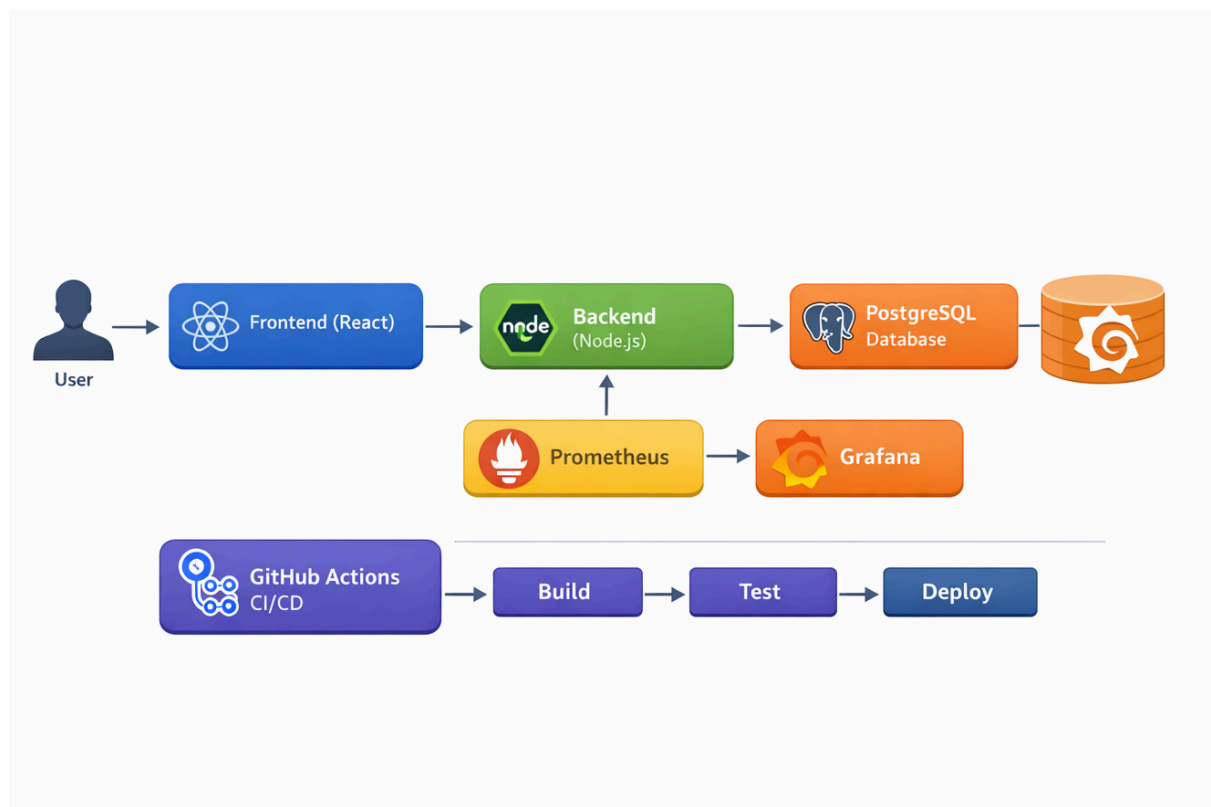


DevOps Starter Kit

Brahima
BAKAYOKO

DevOps Starter Kit PRO – User Guide



1. Introduction

Welcome to **DevOps Starter Kit PRO**.

This project is a **production-ready DevOps boilerplate** designed to help you deploy and operate a fullstack application in minutes.

What you get:

- A complete Dockerized fullstack application
- PostgreSQL database integration
- Observability stack (Prometheus + Grafana)
- CI/CD pipeline ready to use
- Automated deployment scripts

The goal: **go from zero to production-like environment in minutes, not hours.**

2. Architecture Overview

This project follows a modern DevOps architecture:

- Frontend: React (Dockerized)
- Backend: Node.js API
- Database: PostgreSQL
- Monitoring: Prometheus + Grafana
- CI/CD: GitHub Actions
- Containerization: Docker + Docker Compose

Flow overview:

User → Frontend → Backend API → PostgreSQL
Monitoring layer tracks all services in real time.

3. Prerequisites

Before starting, ensure you have installed:

- Docker
- Docker Compose
- Git

Verify installation:

```
docker --version
```

```
docker compose version
```

4. Setup Step-by-Step

1. Clone the project

```
git clone <YOUR_REPO_URL>  
cd devops-starter-kit
```

2. Configure environment

```
cp .env.example .env
```

Edit .env:

```
POSTGRES_USER=devuser  
POSTGRES_PASSWORD=devpass  
POSTGRES_DB=devdb
```

```
DB_HOST=db  
DB_PORT=5432  
DB_USER=devuser  
DB_PASSWORD=devpass  
DB_NAME=devdb
```

5. One-Command Deployment (Core Feature)

This is the **main selling point of the project**.

Run deployment:

```
chmod +x scripts/deploy.sh  
./scripts/deploy.sh
```

What happens automatically:

- Checks Docker installation
- Pulls latest images
- Stops existing containers
- Starts full stack
- Displays running services

This replaces multiple manual commands and reduces errors.

6. Monitoring System

This project includes a **complete observability stack** powered by Prometheus and Grafana.

Access Monitoring Tools

Grafana (Visualization Dashboard)

- URL: <http://localhost:3001>
- Login: admin / admin

Prometheus (Metrics Collection)

- URL: <http://localhost:9090>

Prometheus collects metrics, Grafana visualizes them.

Initial Setup (Required)

1. Add Prometheus as Data Source in Grafana

- Go to **Settings** → **Data Sources**
- Click **Add data source** → **Prometheus**
- Set URL:

`http://prometheus:9090`

Use prometheus (Docker network), not localhost.

2. Import Prebuilt Dashboard

A ready-to-use dashboard is included:

`monitoring/grafana-dashboard.json`

Steps:

- Go to **Dashboards** → **Import**
- Upload the JSON file
- Select **Prometheus** as data source
- Click **Import**

Key Metrics Explained

These metrics give you real-time insights into your application.

Requests per second

```
rate(http_requests_total[1m])
```

Measures incoming traffic

Useful to detect spikes or high load

CPU Usage

```
rate(process_cpu_seconds_total[1m])
```

Shows how much CPU your app is using

Helps detect performance bottlenecks

Memory Usage

```
process_resident_memory_bytes
```

Tracks RAM usage

Useful for identifying memory leaks

Event Loop Lag (Node.js critical metric)

```
nodejs_eventloop_lag_seconds
```

Measures Node.js responsiveness

High values = backend slowdown

How to Use These Metrics (THIS is what sells)

Don't just look at dashboards — **interpret them**

Example scenarios:

- Traffic spike
 - Requests/sec increases suddenly
 - Check CPU & response time
- Slow API
 - High event loop lag

- Backend is overloaded
- Memory issue
 - Memory continuously increases
 - Possible memory leak

Troubleshooting Monitoring

No data in Grafana

- Check Prometheus targets: <http://localhost:9090/targets>
- Ensure backend /metrics endpoint is exposed

Prometheus not reachable

- Verify URL:

`http://prometheus:9090`

7. CI/CD Pipeline

This project includes a **GitHub Actions CI/CD pipeline for advanced users only.**

Important:

You do NOT need CI/CD to run this project.

The application works **out-of-the-box using prebuilt Docker images from Docker Hub.**

Default Usage (Recommended)

This is the simplest and recommended way to run the project.

`./scripts/deploy.sh`

What happens:

- Pulls prebuilt Docker images
- Starts full stack (frontend, backend, database, monitoring)
- No configuration required

Benefits:

- No GitHub setup needed
- No secrets required
- Instant deployment
- Beginner-friendly

Advanced Usage (Optional CI/CD)

This mode is for users who want to **customize or rebuild the project pipeline.**

Steps:

1. Upload the project to your own GitHub repository
2. Configure GitHub Secrets:
 - DOCKER_USERNAME
 - DOCKER_PASSWORD
3. Update image names in the pipeline if you are using a different Docker Hub account

CI/CD Pipeline Location

The pipeline is located in:

`.github/workflows/`

What the Pipeline Does

When triggered, the pipeline:

- Builds backend Docker image
- Builds frontend Docker image
- Pushes images to Docker Hub
- Produces versioned deployment-ready artifacts

Important Notes

- CI/CD is **optional**
- It is designed for **developers who want to extend or customize the project**
- It is NOT required for running or selling the template

Best Practices (Optional for Advanced Users)

If you use CI/CD:

- Keep `.env` for local secrets
- Avoid hardcoding credentials
- Use versioned Docker tags instead of only latest
- Monitor system performance after deployments
- Automate deployments only after successful builds

Summary

No CI/CD needed for standard usage
Prebuilt images available on Docker Hub
CI/CD is an advanced extension feature
Designed for scalability and customization

8. Troubleshooting

View logs

`docker compose logs -f`

Restart services

`docker compose restart`

Reset environment

`docker compose down -v`

`docker compose up -d`

Common issues:

- Port conflicts → change ports in `.env`
- DB connection issues → verify `DB_HOST = db`
- Containers not starting → check logs

9. Production Workflow

Recommended workflow in production-like usage:

`git pull`

`./scripts/deploy.sh`

This ensures consistent and repeatable deployments.

10. Best Practices

- Always use `.env` for secrets
- Keep Docker images updated
- Monitor CPU & memory regularly
- Use CI/CD instead of manual builds
- Automate deployments whenever possible

11. Upgrade Path

This starter kit is designed to scale.

Next steps:

- Move to Kubernetes (K3s / EKS / AKS)
- Add HTTPS with NGINX or Traefik
- Implement authentication (JWT / OAuth)
- Add horizontal scaling
- Deploy to AWS / VPS / Cloud

12. Support & Contact

If you purchased or use this project:

- Leave feedback
- Suggest improvements
- Share your deployment

This helps improve future versions of the DevOps Starter Kit PRO.

End Result

With this project, you now have:

Production-ready fullstack app
Dockerized architecture
Observability stack
CI/CD pipeline
Automated deployment system